Chương 15: Xử lý Chuỗi bằng RNN và CNN

Giảng viên: [Tên Giảng Viên]

Đại Học Đông Á

Ngày 10 tháng 8 năm 2026

Nội dung Chương 15

1. 1. Dữ liệu Chuỗi và Mạng nơ-ron hồi quy (RNN)
2. 2. Ứng dụng và Kiến Trúc Dựa Trên Chuỗi
3. 3. Huấn luyện RNN: BPTT
4. 4. Dự báo Chuỗi Thời Gian (Time Series)
5. 5. Mạng RNN Sâu (Deep RNNs)
6. 6. Kiến trúc Tế bào Nhớ: LSTM và GRU
7. 7. WaveNet (Mạng CNN 1D cho Chuỗi)
8. Tổng kết

Khái niệm: Dữ liệu Chuỗi (Sequence Data) là gì?

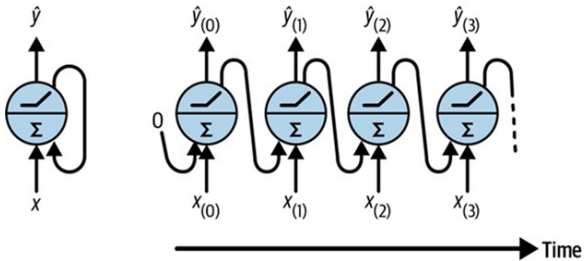
- Trong các chương trước, chúng ta xử lý dữ liệu nơi mỗi điểm dữ liệu là **độc lập và phân phối chuẩn (i.i.d)**.
- Tuy nhiên, trong **dữ liệu chuỗi**, tính thứ tự và sự phụ thuộc thời gian là cốt lõi.
- Ví dụ điển hình:
  - *Chuỗi thời gian kinh tế:* Giá cổ phiếu qua các ngày, GDP hằng năm.
  - *Dữ liệu y tế:* Điện tâm đồ (ECG), điện não đồ (EEG).
  - *Ngôn ngữ tự nhiên (NLP):* Một câu là một chuỗi các từ có cấu trúc ngữ pháp.
  - *Âm thanh / Video:* Chuỗi các khung hình và sóng âm theo thời gian.

Giới hạn của Mạng Truyền thẳng (Feedforward Networks)

- **Mạng truyền thẳng (như MLP, CNN truyền thống)**:
  - Kích thước đầu vào bị cố định (Fixed-size input).
  - Không có bộ nhớ (Memoryless): Mỗi lần nhận đầu vào mới, mạng quên hoàn toàn các đầu vào trước đó.
  - Không nắm bắt được phụ thuộc xa (Long-term dependencies) trừ khi nạp một lượng lớn đầu vào cùng lúc.
- Giải pháp: Cần một mạng có khả năng **lưu giữ trạng thái** của những gì nó đã thấy.

Kiến trúc Mạng RNN Cơ bản

- **Mạng Nơ-ron Hồi quy (RNN - Recurrent Neural Network)** được thiết kế để giải quyết vấn đề này.
- Một nơ-ron hồi quy đơn giản không chỉ nhận đầu vào từ dữ liệu $\mathbf{x}_{(t)}$ tại bước t, mà còn nhận **trạng thái ẩn** $\mathbf{h}_{(t-1)}$ từ bước trước đó.
- Mạng có thể xử lý chuỗi độ dài tùy ý vì nó thực hiện một phép tính lặp lại (recurrent) ở mỗi bước.



Hình 15-1: RNN cơ bản và phiên bản mở rộng

Trạng thái Ẩn (Hidden State) và Tính Toán Toán Học

- Tại mỗi bước thời gian t, nơ-ron RNN (hoặc lớp RNN) tính toán trạng thái tiếp theo như sau:

$$\mathbf{h}_{(t)} = \phi\left(\mathbf{W}_{hx}\mathbf{x}_{(t)} + \mathbf{W}_{hh}\mathbf{h}_{(t-1)} + \mathbf{b}\right)$$

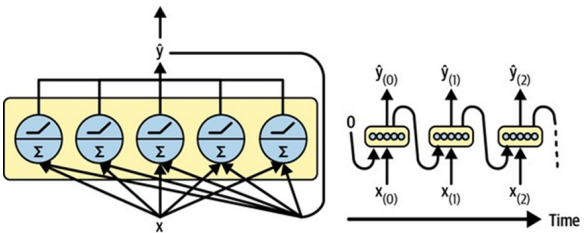
- Trong đó:
  - $\mathbf{x}_{(t)}$: Đầu vào tại bước t.
  - $\mathbf{h}_{(t-1)}$: Trạng thái ẩn từ bước $t-1$ (mang thông tin "trí nhớ").
  - $\mathbf{W}_{hx}$: Trọng số cho đầu vào.
  - $\mathbf{W}_{hh}$: Trọng số cho trạng thái ẩn.
  - $\mathbf{b}$: Vector độ lệch (bias).
  - ϕ: Hàm kích hoạt (thường dùng `tanh` thay vì `ReLU` ở RNN chuẩn để tránh phát nổ giá trị).

Đặc điểm của Trọng Số trong RNN

- **Chia sẻ Trọng số (Weight Sharing):**
- Khác với mạng MLP nơi mỗi lớp có bộ trọng số riêng, RNN sử dụng **cùng một bộ ma trận trọng số** $\mathbf{W}_{hx}, \mathbf{W}_{hh}$ cho tất cả các bước thời gian t.
- Lợi ích:
  - Giảm số lượng tham số đáng kể, tránh quá khớp (overfitting).
  - Có khả năng tổng quát hóa cho các chuỗi có độ dài chưa từng thấy trong tập huấn luyện.

Lớp RNN (RNN Cell Layer)

- Hình 15-2 biểu diễn một **lớp nơ-ron RNN** (RNN Layer) thay vì chỉ một nơ-ron đơn.
- Thay vì nhận một đặc trưng, mạng nhận một vector đặc trưng $\mathbf{x}_{(t)}$.
- Lớp này trả về vector đầu ra $\mathbf{y}_{(t)}$ (thường chính là $\mathbf{h}_{(t)}$).
- Ký hiệu vector giúp mở rộng sang phép tính ma trận cho **Mini-batch** (nhiều chuỗi cùng lúc).



Hình 15-2: Lớp RNN qua thời gian

Phân loại Bài toán xử lý Chuỗi

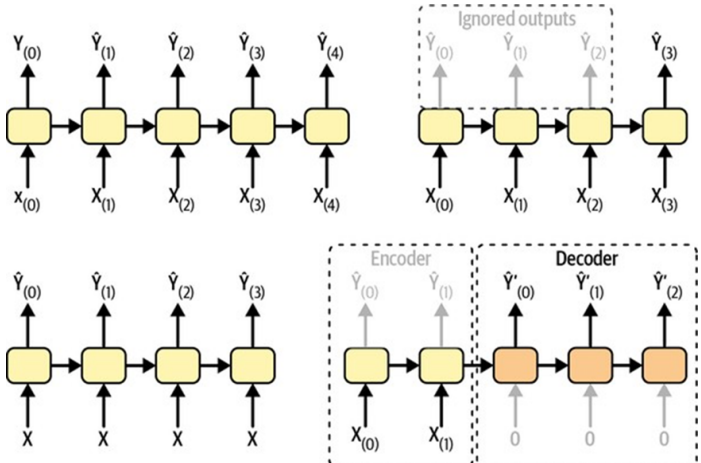
Tùy thuộc vào yêu cầu đầu vào / đầu ra, ta có các loại bài toán:

1. **Sequence-to-Sequence (Chuỗi sang Chuỗi đồng bộ):** Đầu vào N bước, xuất N bước. (Dự báo thời tiết liên tục).
2. **Sequence-to-Vector (Chuỗi sang Vector):** Đầu vào N bước, chỉ xuất 1 kết quả ở cuối chuỗi. (Phân loại văn bản, Sentiment analysis).
3. **Vector-to-Sequence (Vector sang Chuỗi):** Đầu vào 1 giá trị, sinh ra chuỗi độ dài N. (Tạo chú thích ảnh - Image Captioning).
4. **Encoder-Decoder (Delayed Seq-to-Seq):** Nhận chuỗi N, dịch qua chuỗi M. (Dịch máy, Trả lời câu hỏi).

Minh Họa 4 Dạng Kiến Trúc RNN Cốt Lõi

Ghi chú Hình 15-4:

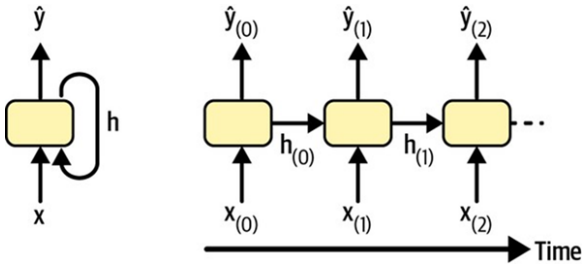
- *Trên trái:* Seq-to-Seq (đồng bộ).
- *Trên phải:* Seq-to-Vec. Đầu ra chỉ quan tâm ở bước cuối.
- *Dưới trái:* Vec-to-Seq. Đầu vào duy nhất ở $t = 0$.
- *Dưới phải:* Mã hóa - Giải mã. Chờ hết câu rồi mới dịch.



Hình 15-4: Các dạng Seq2Seq, Seq2Vec, Vec2Seq

Ứng Dụng: Mạng Chuỗi Sang Chuỗi Dự Báo Trễ

- **Hình 15-3** mô tả kiến trúc dùng nhiều nhất cho Time Series Forecasting (Dự báo chuỗi thời gian).
- Kỳ vọng mạng dự đoán tương lai $y_{(0)}, y_{(1)}$... dựa trên quá khứ $x_{(0)}, x_{(1)}, ...$
- Đầu ra $y_{(t)}$ tại thời điểm t là kết quả dự báo cho $x_{(t+1)}$, do đó mạng bị trễ 1 bước (shifted by 1 step).



Hình 15-3: Seq-to-Seq trễ trong dự báo

Huấn luyện mạng Neural truyền thống (Nhắc lại)

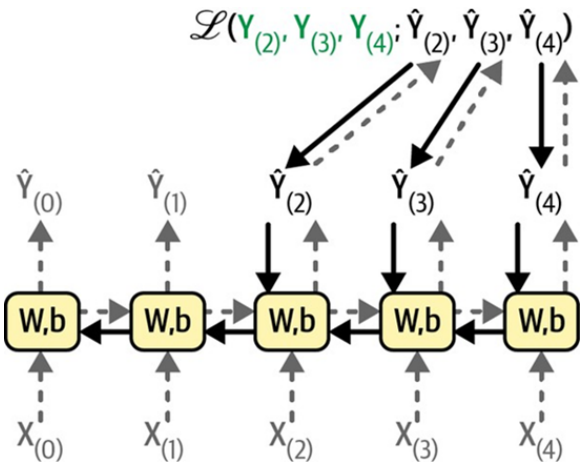
- Mạng truyền thẳng được huấn luyện bằng **Lan truyền ngược (Backpropagation)**:
  1. Lan truyền xuôi (Forward pass) để tính hàm mất mát (Loss).
  2. Tính Gradient (đạo hàm) của Loss theo các trọng số từ lớp cuối về lớp đầu bằng quy tắc chuỗi (Chain rule).
  3. Cập nhật trọng số bằng Gradient Descent.
- Đối với RNN, chuỗi thời gian $t = 1...T$ làm cấu trúc mạng giống như một mạng rất sâu T lớp. Cần một biến thể đặc biệt.

Backpropagation Through Time (BPTT)

- **Lan truyền ngược qua thời gian (BPTT)** là kỹ thuật huấn luyện RNN:
- **Bước 1 (Mở cuộn - Unrolling):** Tạm thời xem RNN như một mạng Feedforward khổng lồ với T lớp (nếu chuỗi dài T).
- **Bước 2 (Lan truyền xuôi):** Tính Loss trên mọi bước thời gian (hoặc chỉ bước cuối).
- **Bước 3 (Lan truyền ngược):** Gradient lan truyền ngược dọc theo chiều thời gian từ $t = T$ về $t = 0$.
- Tổng Gradient cho trọng số mạng là tổng của các Gradient tại mọi bước thời gian.

Cơ chế Hoạt động của BPTT (Minh họa)

- **Hình 15-5** minh họa quá trình BPTT.
- Mũi tên liền nét dày mô tả luồng Gradient ngược từ hàm mất mát L.
- Do $\mathbf{W}$ được chia sẻ ở mọi t, gradient cuối cùng để cập nhật $\mathbf{W}$ là tổng $\sum \nabla \mathbf{W}^{(t)}$.
- Phép tính này cực kỳ đắt đỏ và tốn RAM nếu độ dài chuỗi rất lớn.



Hình 15-5: BPTT - Lan truyền ngược qua thời gian

Vấn Đề: Vanishing Gradients (Suy thoái Gradient)

- **Nguyên nhân:** Trong BPTT, đạo hàm phải được nhân với ma trận trọng số W_{hh} liên tục T lần (qua T bước thời gian).
- $\nabla h_{(0)} \propto (W_{hh})^T \nabla h_{(T)}$
- Nếu giá trị riêng lớn nhất của $W_{hh} < 1$, gradient tiến dần về 0 theo hàm số mũ.
- **Hậu quả:** Mạng không thể học được phụ thuộc xa (Ví dụ: Từ ở đầu câu văn quá dài không có ảnh hưởng gì tới dự đoán ở cuối câu). Trí nhớ mạng bị "ngắn hạn".